Multimedia Signal processing

Prof. Marco Tagliasacchi

February, 11 2013

1. **Question (8 pts)**:
   Consider a continuous source X, whose p.d.f. is given by

$$p_X(x) = \frac{1}{2}e^{-|x|} \tag{1}$$

   Let $\mathcal{Q}$ denote a scalar uniform quantizer with four quantization regions: $I_1 = (-\infty, -1]$, $I_2 = (-1, 0]$, $I_3 = (0, 1]$ and $I_4 = (+1, +\infty)$.
   (a) Compute the entropy, in bits / symbol, of the source representing the output of the quantizer.
   (b) Find the optimal values of the reconstruction levels that minimize the MSE distortion.
   (c) Compute the MSE distortion of the quantizer above (you are not requested to perform the computations exhaustively).
   Hint. You might find the following indefinite integral useful

$$\int ae^{ca}da = \frac{e^{ca}}{c}(ca - 1) \tag{2}$$

$$\int a^2e^{ca}da = e^{ca}\left(\frac{a^2}{c} - \frac{2a}{c^2} + \frac{2}{c^3}\right) \tag{3}$$

   Solution
   (a) The probabilities of the output symbols of the quantizer can be found as follows

$$p_k = \int_{a \in I_k} p_X(a)da \tag{4}$$

   Hence:

$$p_3 = \int_0^1 \frac{e^{-a}}{2}da = \frac{1}{2}\left[-e^{-a}\right]_0^1 = \frac{1}{2}(1 - e^{-1}) \tag{5}$$

$$p_4 = \int_1^{+\infty} \frac{e^{-a}}{2}da = \frac{1}{2}\left[-e^{-a}\right]_1^{+\infty} = \frac{1}{2}e^{-1} \tag{6}$$

   By symmetry, $p_1 = p_4$ and $p_2 = p_3$.
   The value of the entropy can be computed as follows

$$H(\hat{X}) = -\sum_{k=1}^{4} p_k \log_2 p_k \tag{7}$$

(b) The optimal reconstruction levels can be obtained as the centroid of the corresponding region. That is,

$$y_k = \frac{\int_{a\in I_k} a p_X(a) da}{\int_{a\in I_k} p_X(a) da} \tag{8}$$

Hence, we need to compute

$$\int_{a\in I_3} a p_X(a) da = \int_0^1 a e^{-a} da = \frac{1}{2}\left[e^{-a}(-a-1)\right]_0^1 = \frac{1}{2}(1-2e^{-1}) \tag{9}$$

$$\int_{a\in I_4} a p_X(a) da = \int_1^{+\infty} a e^{-a} da = \frac{1}{2}\left[e^{-a}(-a-1)\right]_1^{+\infty} = e^{-1} \tag{10}$$

The reconstruction levels are

$$y_3 = \frac{1-2e^{-1}}{1-e^{-1}} \tag{11}$$

$$y_4 = 2 \tag{12}$$

(c) The MSE distortion can be expressed as

$$D = \sum_{k=1}^{4} \int_{a\in I_k} (y_k - a)^2 \frac{e^{-a}}{2} da \tag{13}$$

$$= 2\left[\int_{a\in I_3} (y_3 - a)^2 \frac{e^{-a}}{2} da + \int_{a\in I_4} (y_4 - a)^2 \frac{e^{-a}}{2} da\right] \tag{14}$$

where

$$\int_{a\in I_3} (y_3 - a)^2 \frac{e^{-a}}{2} da = \int_0^1 y_3^2 \frac{e^{-a}}{2} da + \int_0^1 a^2 \frac{e^{-a}}{2} da - \int_0^1 2y_3 a \frac{e^{-a}}{2} da \tag{15}$$

$$= \frac{y_3^2}{2}\left[-e^{-a}\right]_0^1 + \frac{1}{2}\left[e^{-1}(-a^2-2a-2)\right]_0^1 - y_3\left[e^{-1}(-a-1)\right]_0^1 = \dots \tag{16}$$

Similarly, it is possible to compute the distortion for the region I_4.

2. **Question (5 pts)**

Discuss the design of the perceptual model in audio coding.

Solution

See slide set `MMSP_2.2_Audio coding`.

3. **Question (5 pts)**

(a) Illustrate by means of a pseudocode, motion estimation by block matching, clearly identifying the input and the output (both data and parameters).

(b) Discuss the computational complexity of block matching and the measures that can be adopted to reduce it.

Solution

See slide set `MMSP_3.2_Video_Coding`.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**
   (a) Consider a transform coding system that processes blocks of N input samples.
   - [T] [F] The variances of the transform coefficients $\sigma^2_{y_k}$, $k = 1, \ldots, N$, are always different from each other.
     FALSE. For example, when the input is a stationary memoryless source, the variances are all the same (and no gain is achieved by transform coding).
   - [T] [F] When the optimal transform is used, the transform coefficients are uncorrelated.
     TRUE. The KLT transform has the property of making the transform coefficients uncorrelated.
   - [T] [F] $R_{TC}(D) \leq R_{VQ}(D)$, where $R_{TC}(D)$ and $R_{VQ}(D)$ are, respectively, the operational rate-distortion curves of transform coding and vector quantization.
     FALSE. In general, $R_{TC}(D) \geq R_{VQ}(D)$, since VQ is able to exploit also non-linear statistical dependencies.
   - [T] [F] The number of bits allocated to each transform coefficient is $R_k \propto \log_2 \sigma^2_{y_k}$
     TRUE. Optimal bit allocation leads to an unequal distribution of the bit budget across the transform coefficients.

   (b) Consider lossy coding a source without memory
   - [T] [F] $D_{LM}(R) \leq D_U(R)$, where LM and U refer to, respectively, the Lloyd-Max and uniform quantizer.
     TRUE.
   - [T] [F] In a Lloyd-Max quantizer, the reconstruction levels are equally likely.
     FALSE. In general, they are not equally likely. Thus, they can be entropy coded.
   - [T] [F] $D_{VQ}(R) = D_{SQ}(R)$, where VQ and SQ denote, respectively, vector and scalar quantization.
     FALSE. $D_{VQ}(R) < D_{SQ}(R)$. Vector quantization provides a (marginal) coding gain, also when the source is memoryless.

   (c) Consider a video coding system.
   - [T] [F] Motion vectors are lossy coded.
     FALSE. Motion vector are lossless coded to ensure that both the encoder and the decoder compute exactly the same motion-compensated frame.
   - [T] [F] The average PSNR of prediction residuals increases when increasing the size of the block used in motion estimation/compensation.
     FALSE. The opposite is true. Smaller blocks lead to a more precise estimation of motion.
   - [T] [F] Temporal redundancy is addressed by means of DPCM.
     TRUE. A closed loop predictive scheme is used to exploit temporal redundancy across consecutive frames.

5. **Question (Matlab - 10pts)** Consider the images *image1.png* and *image2.png*.
   For each of them
   (a) Compute the block-wise DCT with blocks of size $N \times N$.
   (b) Compute the block-wise KLT with blocks of size $N \times N$.
   (c) Estimate the transform coding gain in three cases, when $N = 8$.
   - DCT
   - KLT trained on the same image
   - KLT trained on the other image

(d) Show the result as a bar chart and comment.

(e) Let N vary in the set $\{4, 8, 16, 32\}$, and comment the result.

Solution

```
%% 2013_02_11_MMSP_exam

clear all
close all
clc

%% 1) Consider the images 'image1.png' and 'image2.png'
Image1 = double(imread('image1.png'));
Image2 = double(imread('image2.png'));

[Nr1, Nc1] = size(Image1);
[Nr2, Nc2] = size(Image2);


%% 2) Compute the block-wise DCT with blocks of size NxN, N=8
N = 8;

% Image 1
Image1_dct = zeros(N,N,Nr1*Nc1/N^2);
% for each block size
k = 1;
for br = 1:Nr1/N
    for bc = 1:Nc1/N
        % consider a block of size n x n
        block = Image1((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N);
        % compute the DCT
        block_dct = dct2(block);
        Image1_dct(:,:,k) = block_dct;
        k = k+1;
    end
end

v = zeros(N*N,1);
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image1_dct(r,c,:));
    end
end

G_im1_dct = mean(v)/geomean(v);

% Image 2
Image2_dct = zeros(N,N,Nr2*Nc2/N^2);
% for each block size
k = 1;
for br = 1:Nr2/N
    for bc = 1:Nc2/N
        % consider a block of size n x n
        block = Image2((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N);
        % compute the DCT
        block_dct = dct2(block);
        Image2_dct(:,:,k) = block_dct;
        k = k+1;
    end
end

v = zeros(N*N,1);
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image2_dct(r,c,:));
    end
end

G_im2_dct = mean(v)/geomean(v);

%% 3) Compute the block-wise KLT with blocks of size NxN, N=8
%%    KLT trained on the same image
% Image 1
R1 = zeros(N^2,N^2);
for br = 1:Nr1/N
    for bc = 1:Nc1/N
        block = Image1((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N);
        % It is preferable to subtract the mean of each block, but is
        % avoided here for simplicity
        for r = 1:N
            for c = 1:N
                for rr = 1:N
                    for cc = 1:N
                        R1((r-1)*N+c, (rr-1)*N+cc) = R1((r-1)*N+c, (rr-1)*N+cc) + ...
                            block(r,c) * block(rr,cc);
                    end
                end
            end
```

```
        end
    end
end

% Perform KLT transform
[V, ~] = eig(R1);
%V = fliplr(V);
V = V';
Image1_klt_same = zeros(N, N,Nr1*Nc1/N^2);
k = 1;
for br = 1:Nr1/N
    for bc = 1:Nc1/N
        block = Image1((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N)';
        aux = V*block(:);
        Image1_klt_same(:,:,k) = reshape(aux,N,N)';
        k = k+1;
    end
end

v = zeros(N*N,1);
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image1_klt_same(r,c,:));
    end
end

G_im1_klt_same = mean(v)/geomean(v);

% Image 2
R2 = zeros(N^2,N^2);
for br = 1:Nr1/N
    for bc = 1:Nc1/N
        block = Image2((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N);
        for r = 1:N
            for c = 1:N
                for rr = 1:N
                    for cc = 1:N
                        R2((r-1)*N+c, (rr-1)*N+cc) = R2((r-1)*N+c, (rr-1)*N+cc) + ...
                            block(r,c) * block(rr,cc);
                    end
                end
            end
        end
    end
end


% Perform KLT transform
[V, ~] = eig(R2);
%V = fliplr(V);
V = V';
Image2_klt_same = zeros(N, N,Nr2*Nc2/N^2);
k = 1;
for br = 1:Nr2/N
    for bc = 1:Nc2/N
        block = Image2((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N)';
        aux = V*block(:);
        Image2_klt_same(:,:,k) = reshape(aux,N,N)';
        k = k+1;
    end
end

v = zeros(N*N,1);
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image2_klt_same(r,c,:));
    end
end

G_im2_klt_same = mean(v)/geomean(v);

%% 4) Compute the block-wise KLT with blocks of size NxN, N=8
%%    KLT trained on the other image
% Image 1 - Perform KLT transform
[V, ~] = eig(R2);
%V = fliplr(V);
V = V';
Image1_klt_other = zeros(N, N,Nr1*Nc1/N^2);
k = 1;
for br = 1:Nr1/N
    for bc = 1:Nc1/N
        block = Image1((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N)';
        aux = V*block(:);
        Image1_klt_other(:,:,k) = reshape(aux,N,N)';
        k = k+1;
    end
end

v = zeros(N*N,1);
```

```
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image1_klt_other(r,c,:));
    end
end

G_im1_klt_other = mean(v)/geomean(v);

% Image 2 - Perform KLT transform
[V, D] = eig(R1);
%V = fliplr(V);
V = V';
Image2_klt_other = zeros(N, N,Nr2*Nc2/N^2);
k = 1;
for br = 1:Nr2/N
    for bc = 1:Nc2/N
        block = Image2((br-1)*N+1 : br*N, (bc-1)*N+1 : bc*N)';
        aux = V*block(:);
        Image2_klt_other(:,:,k) = reshape(aux,N,N)';
        k = k+1;
    end
end

v = zeros(N*N,1);
for r=1:N
    for c=1:N
        v((r-1)*N+c) = var(Image2_klt_other(r,c,:));
    end
end

G_im2_klt_other = mean(v)/geomean(v);

%% 5) Show the result as a bar chart
figure(1)
subplot(221),imshow(Image1,[]),colormap('gray'),title('Image 1');
subplot(222),imshow(Image2,[]),colormap('gray'),title('Image 2');
subplot(223),bar([1 2 3],20*log10([G_im1_klt_other,G_im1_dct,G_im1_klt_same]),'b');
set(gca,'XTickLabel',{'KLT other', 'DCT', 'KLT same'});
subplot(224),bar([1 2 3],20*log10([G_im2_klt_other,G_im2_dct,G_im2_klt_same]),'b');
set(gca,'XTickLabel',{'KLT other', 'DCT', 'KLT same'});

%% 6) Evaluate the results by modifying the value of N at line 16
%%    Let N vary in the set {4, 8, 16, 32}
% The coding gain increases as N increases, since inter-pixel correlation
% is exploited at longer distances.
% The differences between the different transforms becomes more significant
% for larger block sizes.
```